Python for R Users — Quick Reference

JSM 2026 Workshop

A side-by-side reference for R users transitioning to Python. Material created by Ethan Meyers (Ethan.Meyers@gmail.com)

1. Syntax & Operators

Concept	R	Python
Assignment	`x <- 3` or `x = 3`	`x = 3`
Exponentiation	`2^3`	`2**3`
Integer division	*(none)*	`17 // 5` → `3`
Modulo	`17 %% 5`	`17 % 5`
Logical AND	`&&` or `&`	`and`
Logical OR	`||` or `|`	`or`
Logical NOT	`!`	`not`
Equality	`==`	`==`
Inequality	`!=`	`!=`
Pipe	`|>` or `%>%`	*(method chaining: `df.op1().op2()`)*
Code blocks	`{ }`	Indentation
Comments	`#`	`#`
Print	`cat()` / `print()`	`print()` (required in scripts)

In Python, `^` is **bitwise XOR** — not exponentiation

2. Data Types

Concept	R	Python
Integer	`3L`	`3`
Float	`3.14`	`3.14`
String	`"hello"`	`"hello"`
Boolean	`TRUE` / `FALSE`	`True` / `False`
Null	`NULL`	`None`
Missing	`NA`	*(no base equivalent; Pandas uses `NaN`)*
Check type	`class(x)`	`type(x)`
String interpolation	`glue::glue("The value is {my_var}")`	`f"The value is {my_var}"`
Check for NULL/None	`is.null(x)`	`x is None`

3. Data Structures

Python Lists ↔ R Vectors

Operation	R (vector)	Python (list)
Create	`c(1, 2, 3)`	`[1, 2, 3]`
First element	`v[1]`	`my_list[0]` ← **0-based!**
Last element	`v[length(v)]`	`my_list[-1]`
Slice (2nd–4th)	`v[2:4]` *(includes 4)*	`my_list[1:4]` *(excludes 4)*
Slice with step	`v[seq(1, n, by=2)]`	`my_list[::2]`
Length	`length(v)`	`len(my_list)`
Append	`c(v, x)`	`my_list.append(x)` *(in place)*
Copy	*(copy-on-modify)*	`my_list.copy()` ← required!
Vectorized math	`v + 1`	`my_list + 1` → use NumPy
Named equivalent	*(named vector)*	dictionary `{}`

Python Dictionaries ↔ R Named Lists

Operation	R (named list)	Python (dict)
Create	`my_list <- list(a=1, b=2)`	`d = {"a": 1, "b": 2}`
Access	`my_list$a` or `my_list[["a"]]`	`d["a"]`
Add/update	`my_list$c <- 3`	`d["c"] = 3`
Membership	`"a" %in% names(my_list)`	`"a" in d`
Safe access	returns `NULL`	`d.get("a", default)`

4. Control Flow

Concept	R	Python
For loop	`for (x in v) { ... }`	`for x in my_list:`
Range of integers	`seq(0, n-1)` or `1:n`	`range(n)` → *0 to n - 1*
Index + value	`for (i in seq_along(v)) { x <- v[i] }`	`for i, x in enumerate(my_list):`
Parallel iteration	`mapply(f, v1, v2)`	`for a, b in zip(list1, list2):`
While loop	`while (cond) { ... }`	`while cond:`
If/else if/else	`if () {} else if () {} else {}`	`if:` / `elif:` / `else:`
Inline if	`ifelse(cond, a, b)`	`a if cond else b`
List transform	`sapply(v, f)` / `purrr::map(v, f)`	`[f(x) for x in my_list]`
Filter transform	`Filter(f, v)` / `purrr::keep(v, f)`	`[x for x in my_list if f(x)]`

5. Functions

Concept	R	Python
Define	`f <- function(x, y=0) { ... }`	`def f(x, y=0):`
Return	Last expression (implicit)	`return value` (explicit)
No return	`NULL`	`None`
Multiple return	Returns last value	Returns tuple; unpack with `a, b = f()`
Check missing arg	`missing(x)`	`if x is None:`

Never use a mutable object (list, dict) as a default argument. Use `None` and create inside the function.

6. NumPy ↔ R Vectors

Concept	R	Python
Load package	*Part of base R*	`import numpy as np`
Create	`c(1, 2, 3)`	`np.array([1, 2, 3])`
Sequence	`seq(0, 10, by=2)`	`np.arange(0, 11, 2)`
Evenly spaced	`seq(0, 1, length.out=5)`	`np.linspace(0, 1, 5)`
Zeros	`rep(0, n)`	`np.zeros(n)`
Ones	`rep(1, n)`	`np.ones(n)`
Element-wise math	`v + 1`, `v * 2`	`arr + 1`, `arr * 2`
Math functions	`sqrt(v)`, `log(v)`	`np.sqrt(arr)`, `np.log(arr)`
Boolean filter	`v[v > 3]`	`arr[arr > 3]`
Combined filter	`v[v > 1 & v < 4]`	`arr[(arr > 1) & (arr < 4)]`
Mean / SD	`mean(v)`, `sd(v)`	`np.mean(arr)`, `np.std(arr)`
Mean, NAs removed	`mean(v, na.rm=TRUE)`	`np.nanmean(arr)`
Vectorized conditional	`ifelse(v > 3, "hi", "lo")`	`np.where(arr > 3, "hi", "lo")`
Set a seed	`set.seed(42)`	`rng = np.random.default_rng(42)`
Normal draws	`rnorm(5)`	`rng.normal(0, 1, 5)`
Uniform draws	`runif(5)`	`rng.uniform(0, 1, 5)`
Sample	`sample(x, 3)`	`rng.choice(x, 3, replace=False)`
Matrix	`matrix(v, nrow=2)`	`np.array([[...],[...]])`
Dimensions	`dim(m)`	`m.shape`
Matrix indexing	`m[1, 2]` (1-based)	`m[0, 1]` (0-based)

7. Pandas ↔ dplyr

Operation	R (dplyr)	Pandas
Load package	`library(dplyr)`	`import pandas as pd`
Load CSV	`read_csv("file.csv")`	`pd.read_csv("file.csv")`
First rows	`head(df)`	`df.head()`
Dimensions	`dim(df)`	`df.shape`
Column types	`str(df)`	`df.info()`
Summary stats	`summary(df)`	`df.describe()`
Select column	`df$col`	`df["col"]` (Series)
Select columns	`select(df, a, b)`	`df[["a", "b"]]` (DataFrame)
Filter rows (Boolean indexing)	`df[df$col > val, ]`	`df[df["col"] > val]`
Filter rows	`filter(df, col > val)`	`df.query("col > val")`
Add/modify col	`df$new = expr`	`df["new"] = expr`
Add/modify col	`mutate(df, new = expr)`	`df.assign(new = expr)`
Sort	`arrange(df, col)`	`df.sort_values("col")`
Sort desc	`arrange(df, desc(col))`	`df.sort_values("col", ascending=False)`
Group + summarize	`group_by(df, col) |> summarize(m=mean(val))`	`df.groupby("col")["val"].mean()`
Multiple aggs	`summarize(m=mean(val), n=n())`	`df.agg(["mean","count"])`
Missing check	`is.na(df$col)`	`df["col"].isna()`
Drop missing	`na.omit(df)`	`df.dropna()`
Fill missing	`tidyr::replace_na(df, list(col=0))`	`df.fillna(0)`
Pipe chain	`df |> filter() |> group_by() |> summarize()`	`df.query().groupby().agg()`
Join on column	`left_join(df1, df2, by="key")`	`df1.merge(df2, on="key", how="left")`
Join on Index		`df1.join(df2, how="left")`
Wide → long	`pivot_longer(df, -id)`	`df.melt(id_vars="id")`
Long → wide	`pivot_wider(df, names_from=k, values_from=v)`	`df.pivot(index="id", columns="k", values="v")`
Factor	`factor(x, levels=..., ordered=TRUE)`	`pd.Categorical(x, categories=..., ordered=True)`

8. Visualization

Matplotlib ↔ Base R

Concept	Base R	Matplotlib
Load package	*Part of base R*	`import matplotlib.pyplot as plt`
Figure + axes	`plot(x, y)`	`fig, ax = plt.subplots()`
Line plot	`plot(x, y, type="l")`	`ax.plot(x, y)`
Scatter	`plot(x, y)`	`ax.scatter(x, y)`
Bar chart	`barplot(heights)`	`ax.bar(labels, heights)`
Histogram	`hist(v)`	`ax.hist(v, bins=20)`
X label	`xlab("label")`	`ax.set_xlabel("label")`
Y label	`ylab("label")`	`ax.set_ylabel("label")`
Title	`main="title"`	`ax.set_title("title")`
Legend	`legend(...)`	`ax.legend()`
Show	*(automatic)*	`plt.show()`

Seaborn ↔ ggplot2

Seaborn figure-level functions are the closest Python analog to ggplot2; they accept `data=`, `x=`, `y=`, `hue=`, `col=`, `row=`

Task	ggplot2	Seaborn
Load package	`library(ggplot2)`	`import seaborn as sns`
Distribution	`geom_histogram()` / `geom_density()`	`sns.displot(kind="hist"/"kde"/"ecdf")`
Scatter	`geom_point()`	`sns.relplot(kind="scatter")`
Line	`geom_line()`	`sns.relplot(kind="line")`
Bar	`geom_bar()` / `geom_col()`	`sns.catplot(kind="bar")`
Box plot	`geom_boxplot()`	`sns.catplot(kind="box")`
Violin	`geom_violin()`	`sns.catplot(kind="violin")`
Facet	`facet_wrap(~col)`	`col="col_name"` argument
Color by group	`aes(color=group)`	`hue="group"` argument

9. Statistical Modeling

Linear Regression

Concept	R	Python
Load package	*Part of base R*	`import statsmodels.formula.api as smf`
Fit OLS	`lm(y ~ x, data=df)`	`smf.ols("y ~ x", data=df).fit()`
Summary	`summary(model)`	`model.summary()`
Coefficients	`coef(model)`	`model.params`
P-values	*(in summary)*	`model.pvalues`
Confidence intervals	`confint(model)`	`model.conf_int()`
R^2	`summary(model)$r.squared`	`model.rsquared`

Distributions & Tests (scipy.stats)

With `from scipy import stats`, every distribution object has the same four operations:

Goal	R	scipy.stats
Load package	*Part of base R*	`from scipy import stats`
Density	`dnorm(x)`	`stats.norm.pdf(x)`
CDF	`pnorm(x)`	`stats.norm.cdf(x)`
Quantile	`qnorm(p)`	`stats.norm.ppf(p)`
Random draws	`rnorm(n)`	`stats.norm.rvs(size=n)`
Two-sample t-test	`t.test(x, y)`	`stats.ttest_ind(x, y)`

The same pattern applies to `stats.t`, `stats.chi2`, `stats.binom`, and many more.

scikit-learn Workflow

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression   # or any other model
from sklearn.metrics import mean_squared_error

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LinearRegression()
model.fit(X_train, y_train)          # train

y_pred = model.predict(X_test)       # predict
r2     = model.score(X_test, y_test)  # evaluate
rmse   = mean_squared_error(y_test, y_pred)**0.5
```

The same three-step API (`fit`, `predict`, `score`) works for any sklearn estimator.

10. Key Python Pitfalls for R Users

Pitfall	What R does	What Python does
`2^3`	`8` (exponentiation)	`1` (bitwise XOR!) — use `2**3`
`my_list + 1`	`c(1,2,3)+1` → `2,3,4`	`TypeError` — use NumPy
`b = a` (list)	Creates independent copy	Both point to the **same** list — use `a.copy()`
`TRUE/FALSE`	Case-insensitive	Must be `True`/`False` exactly
`NULL` / `NA`	Distinct concepts	`None` (no `NA` in base Python)
Implicit return	Last expression returned	Must use `return` explicitly
1-based index	`v[1]` is first element	`my_list[0]` is first element
Slice endpoint	`v[1:3]` includes index 3	`my_list[0:3]` excludes index 3
`and` / `or`	`&&` / `||`	`and` / `or` (symbols are bitwise)
Mutable default	Copy-on-modify protects you	`def f(x, my_list=[])` persists across calls — use `None`
`for x in obj:`	Works on any vector	Requires `__getitem__` (duck typing)